Collections Java et tableaux dynamiques

Cours et exemples pour les étudiants – module POO Java

Auteur : EL MAALMI BILLAL

À retenir : un tableau Java a une taille fixe. Une collection comme `ArrayList` grandit toute seule : c'est un **tableau dynamique**. Tous les exemples de ce document ont été compilés et exécutés ; les résultats affichés sont les vrais.

1. Pourquoi les collections ?

Dans le TP bancaire, la classe `Bank` doit stocker un nombre de comptes **qu'on ne connaît pas à l'avance** : on en crée, on en supprime, on les trie. Avec un tableau classique, il faudrait gérer la taille, les trous et les copies à la main. Les collections du package `java.util` font ce travail pour vous.

2. Le tableau classique : taille fixe

```
int[] notes = new int[3];
notes[0] = 12;
notes[1] = 15;
notes[2] = 9;
System.out.println(Arrays.toString(notes));
System.out.println("Taille fixe : " + notes.length);
// notes[3] = 18;  -> ArrayIndexOutOfBoundsException
```

```
Resultat :
[12, 15, 9]
Taille fixe : 3
```

La taille est décidée à la création et ne change plus. Écrire hors limites lève une `ArrayIndexOutOfBoundsException`.

3. Le tableau dynamique : ArrayList

`ArrayList<T>` offre les opérations d'un tableau, sans limite de taille. Le type entre chevrons (`<String>`) est un **générique** : la liste ne contient que des objets de ce type. On déclare de préférence avec l'interface : `List<String> noms = new ArrayList<>();`

```
List<String> noms = new ArrayList<>();
noms.add("Sara");
noms.add("Omar");
noms.add("Lina");
noms.add(1, "Yassine");            // insere a l'index 1
System.out.println(noms);
System.out.println(noms.get(2));   // lecture par index
noms.set(0, "Salma");              // remplace l'element 0
noms.remove("Omar");               // supprime par valeur
noms.remove(0);                    // supprime par index
System.out.println(noms + " taille = " + noms.size());
System.out.println(noms.contains("Lina"));
System.out.println(noms.indexOf("Lina"));
```

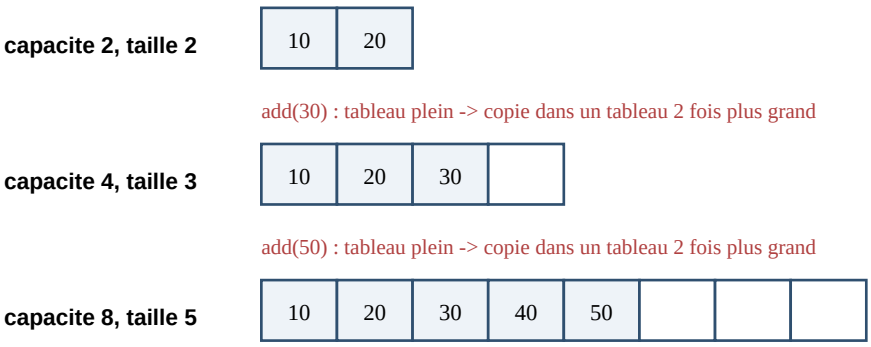
```
Resultat :
[Sara, Yassine, Omar, Lina]
Omar
[Yassine, Lina] taille = 2
true
1
```

Méthode	Effet
`add(x)`	ajoute à la fin
`add(i, x)`	insère à l'index *i* (décale la suite)
`get(i)`	lit l'élément d'index *i*
`set(i, x)`	remplace l'élément d'index *i*
`remove(i)` / `remove(x)`	supprime par index / par valeur
`size()`	nombre d'éléments
`contains(x)`, `indexOf(x)`	recherche (parcours de la liste)
`isEmpty()`, `clear()`	test de vacuité, vidage

4. Comment fonctionne un tableau dynamique ?

Un `ArrayList` cache un **tableau ordinaire** et deux informations : sa **capacité** (la taille du tableau interne) et sa **taille** (le nombre d'éléments réellement stockés). Quand on ajoute un élément alors que le tableau est plein :

- un **nouveau tableau plus grand** est créé (l'implémentation OpenJDK multiplie la capacité par environ 1,5 ; notre exemple double pour simplifier) ;
- les anciens éléments y sont **copiés** ;
- le nouvel élément est ajouté, et l'ancien tableau est abandonné.



Voici une version simplifiée d'un tableau dynamique d'entiers écrite à la main :

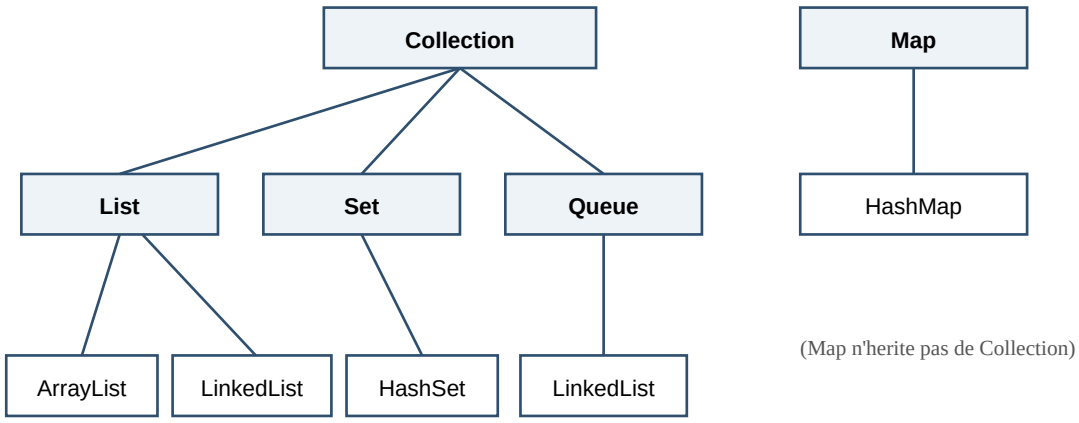
```
static class MyDynamicArray {
    private int[] data = new int[2];   // capacite initiale = 2
    private int size = 0;              // nombre d'elements reellement stockes

    void add(int value) {
        if (size == data.length) {     // tableau plein : on l'agrandit
            data = Arrays.copyOf(data, data.length * 2);
            System.out.println("  agrandissement : capacite = " + data.length);
        }
        data[size++] = value;
    }

    int get(int index) {
        if (index < 0 || index >= size) {
            throw new IndexOutOfBoundsException("Index " + index + ", taille " + size);
        }
        return data[index];
    }

    int size() { return size; }
}
```

```
MyDynamicArray t = new MyDynamicArray();
for (int i = 1; i <= 5; i++) {
    t.add(i * 10);
    System.out.println("ajout de " + (i * 10) + " -> taille = " + t.size());
}
System.out.println("t.get(2) = " + t.get(2));
```

```
Resultat :
ajout de 10 -> taille = 1
ajout de 20 -> taille = 2
  agrandissement : capacite = 4
ajout de 30 -> taille = 3
ajout de 40 -> taille = 4
  agrandissement : capacite = 8
ajout de 50 -> taille = 5
t.get(2) = 30
```

Pourquoi c'est efficace : l'agrandissement coûte cher (copie de tous les éléments) mais arrive rarement, car la capacité double à chaque fois. En moyenne, `add` à la fin est donc en temps constant : on parle de coût **amorti O(1)**.

5. La famille des collections



Interface	Particularité	Implémentation courante
`List`	ordonnée, accès par index, doublons autorisés	`ArrayList`, `LinkedList`
`Set`	aucun doublon, pas d'index	`HashSet`
`Queue`	file : premier arrivé, premier sorti (FIFO)	`LinkedList`
`Map`	associe une **clé** unique à une **valeur**	`HashMap`

6. Exemples : Queue, Set et Map

6.1 Queue (file d'attente)

```
Queue<String> file = new LinkedList<>();
file.add("Client 1");
file.add("Client 2");
file.add("Client 3");
System.out.println(file.poll());   // retire et retourne le premier
System.out.println(file.peek());   // lit le premier sans le retirer
System.out.println(file);
```

```
Resultat :
Client 1
Client 2
[Client 2, Client 3]
```

6.2 Set (ensemble sans doublon)

```
Set<String> villes = new HashSet<>();
villes.add("Meknes");
villes.add("Fes");
villes.add("Meknes");              // doublon ignore
System.out.println(villes.size());
System.out.println(villes.contains("Fes"));
```

```
Resultat :
2
true
```

6.3 Map (clé → valeur)

```
Map<Integer, String> comptes = new HashMap<>();
comptes.put(1, "Sara");
comptes.put(2, "Omar");
comptes.put(1, "Salma");           // meme cle : la valeur est remplacee
System.out.println(comptes.get(1));
System.out.println(comptes.containsKey(3));
for (Map.Entry<Integer, String> e : comptes.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
```

```
Resultat :
Salma
false
1 -> Salma
2 -> Omar
```

7. Trier une liste

Pour trier selon un critère (par exemple le solde dans `Bank.list()`), on passe un `Comparator` à `sort` :

```
List<Compte> comptes = new ArrayList<>();
comptes.add(new Compte(1, 300f));
comptes.add(new Compte(2, 100f));
comptes.add(new Compte(3, 200f));
comptes.sort(Comparator.comparingDouble(Compte::getSolde));
System.out.println(comptes);
```

```
Resultat :
[Compte2(100.0), Compte3(200.0), Compte1(300.0)]
```

8. Pièges classiques

```java
List<Integer> nombres = new ArrayList<>(Arrays.asList(5, 10, 15, 20));

// 1) remove(int) supprime par INDEX, remove(Object) par VALEUR
nombres.remove(1);                      // supprime l'index 1 (la valeur 10)
nombres.remove(Integer.valueOf(20));    // supprime la valeur 20
System.out.println(nombres);

// 2) Supprimer dans un for-each provoque une exception
List<Integer> valeurs = new ArrayList<>(Arrays.asList(5, 10, 15, 20));
try {
    for (Integer v : valeurs) {
        if (v == 10) valeurs.remove(v);
    }
} catch (ConcurrentModificationException e) {
    System.out.println("Erreur : " + e.getClass().getSimpleName());
}

// 3) Solution : removeIf (ou un Iterator)
valeurs.removeIf(v -> v == 10);
System.out.println(valeurs);
```

```
Resultat :
[5, 15]
Erreur : ConcurrentModificationException
[5, 15, 20]
```

Règle : ne modifiez pas une liste pendant que vous la parcourez avec un *for-each*. Utilisez `removeIf` ou un `Iterator`.

9. Choisir la bonne collection

Collection	Lecture par index	Ajout à la fin	Insertion / suppression au milieu	Recherche (contains)
`ArrayList`	O(1)	O(1) amorti	O(n)	O(n)
`LinkedList`	O(n)	O(1)	O(1) si on est déjà positionné, sinon O(n)	O(n)
`HashSet` / `HashMap`	–	O(1) en moyenne	O(1) en moyenne	O(1) en moyenne

Par défaut, utilisez `ArrayList` pour une liste, `HashSet` quand les doublons sont interdits, et `HashMap` pour retrouver une valeur par sa clé.

10. Application au TP bancaire

- Les comptes de la classe `Bank` sont stockés dans une `List<BankAccount>` (un `ArrayList`).
- `create` fait un `add` après avoir vérifié que le compte n'existe pas.
- `search(num)` parcourt la liste avec un for-each et compare les numéros de compte.
- `list()` trie une copie de la liste avec un `Comparator` sur la balance.
- `delete` utilise `remove(account)` (suppression par objet).
- Une `Map<Integer, BankAccount>` (numéro → compte) rendrait `search` en O(1) : c'est une amélioration possible.

Exercices

1. Créez une `ArrayList<Integer>` de 10 notes, puis affichez la moyenne, la note maximale et la liste triée.

2. Retirez les doublons d'une liste de prénoms en utilisant un `HashSet`.

3. Comptez le nombre d'occurrences de chaque mot d'une phrase avec une `HashMap<String, Integer>`.

4. Ajoutez à `MyDynamicArray` une méthode `remove(int index)` qui décale les éléments suivants.